

APCS程式設計班

Lesson8陣列運用 & 函式

Start



講師：王俊斌

時間：每週二19:30-21:30



For

```
for ( 初始變數; 判斷式; 遞增式 ) {  
    陳述句一;  
    陳述句二;  
}
```



陣列(Array)

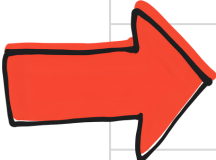
- 什麼是陣列 (Array)
 - 陣列是一種可以一次存放很多**相同型態資料**的資料結構。





您好，很高興為您服務



資料結構	英文	特點	生活例子
陣列	Array	固定大小、存取速度快	一排置物櫃
 向量	Vector	大小可自動增加	可伸縮的收納盒
串列	Linked List	插入、刪除方便	一節一節相連的火車
堆疊	Stack	後進先出 (LIFO)	疊盤子
佇列	Queue	先進先出 (FIFO)	排隊買飲料
雙端佇列	Deque	前後都能加入或刪除	可從兩端拿書的書架
樹	Tree	階層式結構	家族樹、資料夾
二元搜尋樹	Binary Search Tree	快速搜尋資料	排序好的分類架
堆積	Heap	快速取得最大或最小值	排行榜
雜湊表	Hash Table	快速查找資料	字典、電話簿
圖	Graph	描述節點之間的關係	地圖、社群好友

陣列的優缺點

優點：

- ✓ 存取速度快
- ✓ 使用索引方便
- ✓ 記憶體連續
- ✓ 很適合固定大小資料

缺點：

- ✗ 大小固定
- ✗ 插入資料不方便
- ✗ 刪除資料需要搬移元素



向量(vector)

- 什麼是向量(vector)
 - 傳統陣列 (Array) 的痛點：大小固定、無法動態擴增、容易越界。
 - Vector 的優勢：動態長度、內建豐富功能、記憶體自動管理。





都坐下 基本操作



向量的基本操作

1. 引入標頭檔：`#include <vector>`
2. 宣告與初始化：`vector<int> nums;` 或
`vector<int> nums(5, 0);`
3. 尾端尾隨與刪除：`push_back()`、`pop_back()`
4. 長度與清空：`size()`、`empty()`、`clear()`
5. 元素存取：`nums[i]` 與 `nums.at(i)` 的差別。



函式(Function)

- 什麼是函式(Function)
 - 程式碼重用性 (Reusability)、拆解複雜問題 (Divide and Conquer)。
 - 函式基本結構





函式傳遞

- 值傳遞 (Pass by Value) : `void printVector(vector<int> vec)` – 會複製整份資料，效能較低。
- 引用傳遞 (Pass by **Reference**) : `void modify(vector<int> &vec)` – 可以修改原 `vector`，省去複製開銷。
- 常數引用傳遞 (Pass by Const Reference) : `void display(const vector<int> &vec)` – 只讀取不修改)。





Problem-1 陣列反轉與元素倍增

請撰寫一個函式 `processVector`，接收一個整數 `vector`。該函式需要將 `vector` 中的每一個元素都乘以 2，並將整個 `vector` 反轉過來。請在 `main` 函式中讀取輸入，呼叫該函式後印出最終結果。

輸入

第一行包含一個整數 $N (1 \leq N \leq 100)$ ，代表 `vector` 的長度。

第二行包含 N 個整數。

輸出

印出處理後的整數序列，數字之間以空格分隔。。



Problem-2 偶數過濾

請撰寫一個函式 `filterEvens`，接收一個 `const vector<int>&`，並回傳一個新的 `vector<int>`，裡面僅包含原 `vector` 中的所有偶數，且保持原本的相對順序。

輸入

第一行包含一個正整數 N ($1 \leq N \leq 1000$)

第二行包含 N 個整數 ($0 \leq N \leq 10000$)

輸出

請輸出兩行：

印出過濾後的偶數序列，數字之間以空格分隔。若沒有偶數，請印出 `None`。



Problem-3 去除重複元素並排序

請撰寫一個函式 `getUniqueSorted`，接收一個整數 `vector`，回傳一個已排序且不包含任何重複元素的新 `vector`。

輸入

第一行包含一個正整數 N ($1 \leq N \leq 100$)。

第二行包含 N 個整數 (可能包含重複值)。

輸出

印出處理後的序列，數字之間以空格分隔。





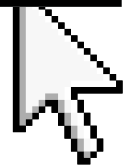
What have we learned...

向量 (vector)

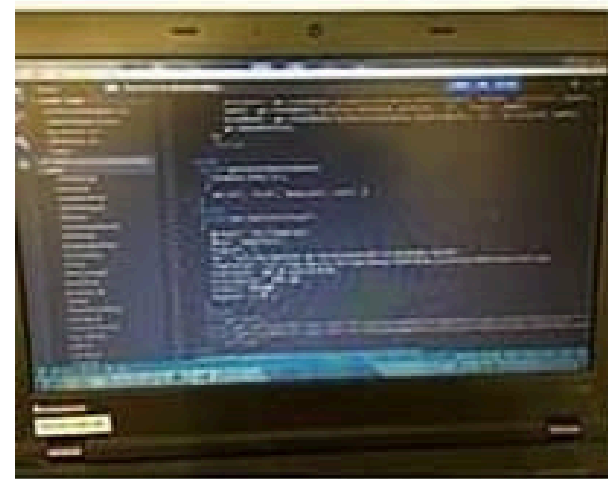
函式

reference

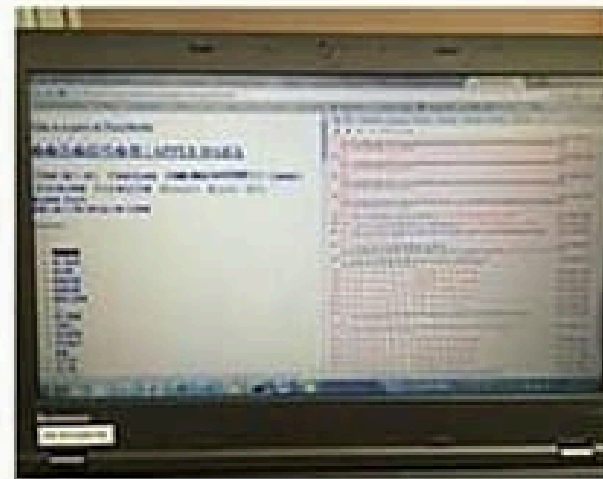
上週題型檢討



如何寫程式



你打了一些程式碼



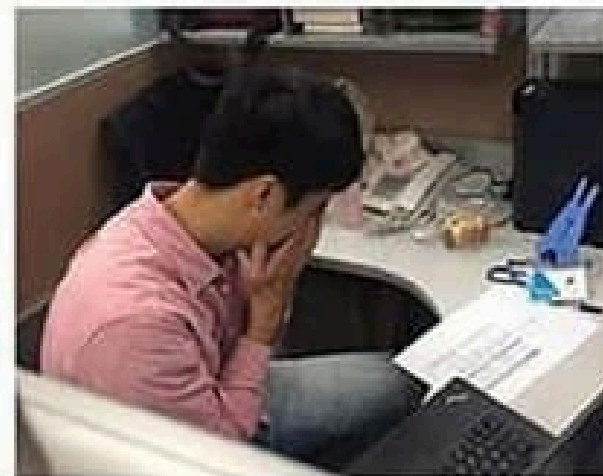
然後你試著執行



你搞砸了,
就像你做其他事情一樣



你的程式碼根本
就是你的人生寫照



充滿bug,可讀性差...



...而且沒有人愛你

